Proyecto Integrador - Fase 1-2

Sistema: Chat Empresarial (Cliente-Servidor en Go con FastAPI)

Darwin Alcarraz, José Almeida, Cayetano Córdova

Facultad de Ciencias Técnicas, Universidad Internacional Del Ecuador

03-SIN-A: Programación Orientada a Objetos

Ing. Ivan Galo Reyes Chacon

5 de octubre de 2026

1. Problema organizacional

Hoy en día muchas empresas pequeñas y equipos de trabajo se comunican por WhatsApp, Telegram o los chats personales que cada uno tenga. Es fácil de usar, pero la empresa no controla nada de lo que pasa ahí. Un informe de McKinsey calcula que un trabajador que depende de hablar con otros pierde casi el 20 % de su semana buscando información interna o tratando de encontrar a quién preguntarle (Chui et al., 2012). Y en una encuesta en Reino Unido, 1 de cada 3 empleados dijo que usa apps como WhatsApp o Telegram para comunicarse en el trabajo (Guthkelch, 2025). Son datos de otros países, pero sirven para ver que el problema es real.

Usar estos chats trae varios problemas:

- **Información dispersa:** las decisiones y los acuerdos quedan en el celular de cada persona, y después cuesta encontrarlos.
- **Datos poco protegidos:** la empresa no sabe quién puede ver las conversaciones ni qué pasa si alguien pierde su teléfono.
- **Falta de registro:** no hay forma de saber cuántos mensajes se envían ni cuánto tiempo está conectada la gente.

Solución. Para resolverlo proponemos un chat empresarial interno y cerrado. Funciona con un cliente en Go, un servidor de chat también en Go que guarda los mensajes, y un servicio de autenticación en FastAPI que crea y revisa el token de sesión. El usuario puede iniciar sesión, enviar mensajes, ver el historial y cerrar sesión, y cada una de estas acciones se guarda con el usuario, la fecha y la hora. Con eso después se pueden sacar datos como mensajes por usuario y por hora, sesiones por día, tiempo promedio de sesión e intentos fallidos de inicio de sesión.

Por qué el chat. Escogimos el chat porque es la opción que mejor cumple lo que pide el proyecto: resuelve un problema claro, tiene operaciones cliente-servidor reales y genera datos que se pueden medir. Los correos internos se parecen, pero el control de sesiones tiene menos peso, y con imágenes o videos habría que manejar archivos pesados, que es más complicado para una primera versión.

Alcance. En esta versión el chat incluye inicio y cierre de sesión con token, mensajes de texto entre dos usuarios, historial por páginas y el registro de datos de uso. Para recibir mensajes, el cliente le pregunta al servidor cada pocos segundos si hay algo nuevo. Dejamos fuera el envío de archivos, los chats de grupo, las notificaciones, el registro de usuarios nuevos desde la app y cualquier servicio externo.

2. Requisitos

Requisitos funcionales

ID	Categoría	Requisito
RF-01	Autenticación y sesiones	Autenticación de usuarios: el usuario registrado inicia sesión con su usuario y contraseña. FastAPI revisa los datos y entrega un token que vence.
RF-02	Mensajería	Envío y recepción de mensajes: el usuario envía mensajes de texto a otro usuario desde el cliente en Go. El otro usuario los recibe porque su cliente consulta al servidor cada cierto tiempo.
RF-03	Mensajería	Consulta de historial: se pueden ver los mensajes anteriores de una conversación, por páginas y ordenados por fecha.
RF-04	Autenticación y sesiones	Control de sesiones: se guarda cuándo empieza y cuándo termina cada sesión. Al cerrar sesión el token deja de servir y los tokens vencidos se rechazan.
RF-05	Estadísticas	Registro de datos de uso: cada acción (mensaje, inicio o cierre de sesión, intento fallido, consulta de historial) se guarda con usuario, fecha y hora.

Requisitos no funcionales

ID	Categoría	Requisito
RNF-01	Rendimiento	Tiempo de respuesta: una página del historial (hasta 50 mensajes) debe cargar en menos de 2 segundos. El cliente en Go no se debe congelar mientras espera al servidor (se usan goroutines).
RNF-02	Seguridad	Acceso con token: toda petición de mensajes o historial lleva un token, que el servidor de chat revisa con FastAPI.
RNF-03	Portabilidad	Multiplataforma: el cliente en Go debe correr en Linux, Windows y macOS.

RNF-04	Seguridad	Contraseñas: se guardan cifradas (con hash), nunca en texto plano.
RNF-05	Interoperabilidad	API REST: el cliente, el servidor de chat y FastAPI se comunican por HTTP con datos en JSON.

3. Casos de uso

Describimos cuatro casos de uso con el mismo formato. Los tres primeros son los principales y el cuarto completa el control de sesiones.

CU-01: Iniciar sesión

Actores: usuario (empleado) y servicio FastAPI.

Precondiciones: el usuario ya está registrado y los servicios están funcionando.

Flujo básico

1. El usuario escribe su usuario y contraseña en el cliente.
2. El cliente manda los datos a FastAPI (POST /auth/login).
3. FastAPI compara la contraseña con el hash guardado, crea un token con fecha de vencimiento y anota el inicio de la sesión.
4. El cliente recibe el token y abre la pantalla del chat.

Flujos alternativos

- **A1 (paso 3). Credenciales incorrectas:** FastAPI responde con error 401 y anota el intento fallido. El cliente muestra "Usuario o contraseña incorrectos".
- **A2 (paso 2). FastAPI no responde:** el cliente avisa que no se pudo conectar y no abre el chat.

Postcondiciones: el usuario queda autenticado con su token y el inicio de sesión queda guardado con fecha y hora.

Datos medibles: sesiones iniciadas por día e intentos fallidos.

CU-02: Enviar mensaje

Actores: usuario autenticado, servidor de chat y servicio FastAPI.

Precondiciones: el usuario tiene una sesión activa con token vigente y ya eligió con quién hablar.

Flujo básico

1. El usuario elige al destinatario, escribe el mensaje y presiona enviar.
2. El cliente manda el mensaje al servidor de chat (POST /api/mensajes) con el token.
3. El servidor le pide a FastAPI que revise el token (GET /auth/validar).
4. FastAPI confirma que el token sirve y de qué usuario es.
5. El servidor guarda el mensaje en la base de datos y actualiza las métricas.
6. El cliente muestra el mensaje enviado. El cliente del destinatario lo recibe en su siguiente consulta (GET /api/mensajes/nuevos).

Flujos alternativos

- **A1 (paso 1). Mensaje vacío:** el cliente no lo envía.
- **A2 (paso 4). Token inválido o vencido:** el servidor responde 401, el cliente avisa y vuelve al inicio de sesión (CU-01). El mensaje no se guarda.
- **A3 (paso 5). El destinatario no existe:** el servidor responde 404 y el cliente avisa. El mensaje no se guarda.

Postcondiciones: el mensaje queda guardado con remitente, destinatario, texto, fecha y hora, y se suma a las métricas de volumen.

Datos medibles: mensajes por usuario, mensajes por hora del día y tiempo de respuesta al enviar.

CU-03: Consultar historial

Actores: usuario autenticado, servidor de chat y servicio FastAPI.

Precondiciones: el usuario tiene una sesión activa con token vigente y abrió una conversación.

Flujo básico

1. El usuario abre una conversación en el cliente.
2. El cliente pide el historial al servidor de chat (GET /api/historial?con=usuario&pagina=1) con el token.
3. El servidor revisa el token con FastAPI (GET /auth/validar).
4. El servidor busca los mensajes entre los dos usuarios, los ordena por fecha y los devuelve en JSON.
5. El cliente los muestra con remitente, texto y fecha y hora, y el servidor guarda que se hizo la consulta.

Flujos alternativos

- **A1 (paso 3). Token inválido o vencido:** el servidor responde 401 y el cliente vuelve al inicio de sesión (CU-01). No se muestra nada.

- **A2 (paso 4). Sin mensajes:** el servidor devuelve una lista vacía y el cliente muestra "Aún no hay mensajes".
- **A3 (paso 5). Más de una página:** al llegar al final de la lista, el cliente pide la página siguiente.

Postcondiciones: el cliente muestra los mensajes ordenados por fecha sin modificarlos y la consulta queda registrada.

Datos medibles: consultas de historial por usuario, mensajes por consulta y tiempo de respuesta.

CU-04: Cerrar sesión

Actores: usuario autenticado y servicio FastAPI.

Precondiciones: el usuario tiene una sesión activa.

Flujo básico

1. El usuario presiona "Cerrar sesión".
2. El cliente manda la solicitud a FastAPI (POST /auth/logout) con el token.
3. FastAPI invalida el token, anota la hora de cierre y calcula cuánto duró la sesión.
4. El cliente borra el token y vuelve a la pantalla de inicio de sesión.

Flujos alternativos

- **A1 (paso 3). Token ya vencido:** FastAPI cierra la sesión con la hora en que venció.
- **A2 (paso 2). FastAPI no responde:** el cliente borra su token y vuelve al inicio de sesión. La sesión se cierra en el servidor cuando el token venza.

Postcondiciones: el token deja de servir y quedan guardadas la hora de cierre y la duración.

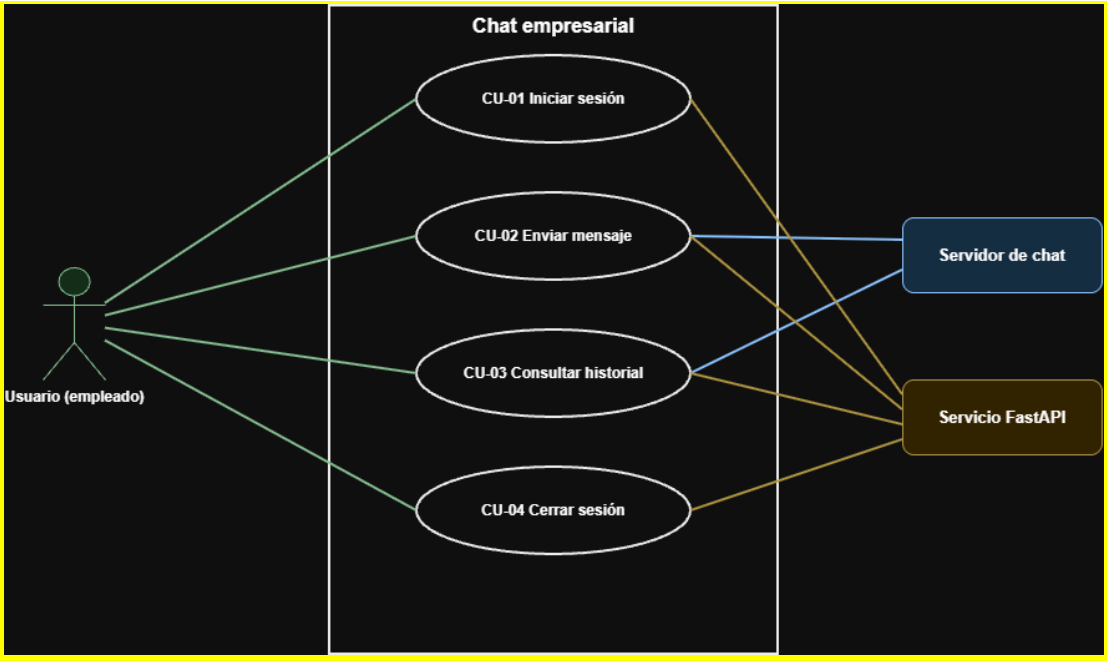
Datos medibles: duración de cada sesión y tiempo promedio de sesión.

4. Matriz de trazabilidad

La matriz relaciona cada requisito con su caso de uso y muestra qué hace cada componente: el cliente y el servidor de chat en Go, y el servicio de autenticación en FastAPI. Así se comprueba que ningún requisito queda sin cubrir y de dónde sale cada dato medible.

Requisito	Caso de uso	Go (cliente y servidor de chat)	FastAPI	Datos medibles
RF-01 Autenticación	CU-01	El cliente manda usuario y contraseña y guarda el token.	Revisa las credenciales y crea el token.	Sesiones por día, intentos fallidos.
RF-02 Mensajería	CU-02	El cliente envía y consulta mensajes nuevos; el servidor los guarda.	Valida el token.	Mensajes por usuario, mensajes por hora.
RF-03 Historial	CU-03	El cliente lo pide y lo muestra; el servidor busca y devuelve los mensajes.	Valida el token.	Consultas de historial, mensajes por consulta.
RF-04 Control de sesiones	CU-01, CU-04	El cliente borra el token al cerrar sesión.	Guarda inicio y fin de sesión e invalida el token.	Duración de sesión, tiempo promedio.
RF-05 Estadísticas	CU-01 a CU-04	El servidor guarda cada mensaje y consulta con usuario, fecha y hora.	Guarda sesiones e intentos fallidos.	Todas las métricas anteriores.
RNF-01 a RNF-05	CU-01 a CU-04	Goroutines, multiplataforma, token en cada petición, REST con JSON.	Hash de contraseñas, validación del token, REST con JSON.	Tiempo de respuesta.

5. Diagrama general de casos de uso



Conclusiones

A partir del análisis presentado, el desarrollo y especificación de la arquitectura para el Sistema de Chat Empresarial Interno ataca los riesgos operativos y de seguridad que pueden tener su origen en la utilización de aplicaciones de mensajería de carácter informal o personal (WhatsApp, Telegram, …) en la empresa. La separación de responsabilidades de forma explícita entre un cliente y un servidor central en Go y un servicio de autenticación en FastAPI, permite enmarcar el perímetro de control corporativo sobre la comunicación y la custodia de los datos.

La arquitectura basada en tokens de sesión con caducidad muy estricta y la transmisión cliente-servidor a través de una API REST/JSON asegura la transmisión de datos externa en cuanto a la seguridad,

trazabilidad e interoperabilidad. Y a su vez un registro centralizado de auditoría (logs de utilización, métricas de sesión, volumen de mensajes e intentos erróneos) no solo da cumplimiento a requisitos funcionales de mensajería e historial paginado, sino que la organización puede dotarse de capacidad analítica de datos operativa o de monitoreo de accesos en tiempo real. Esta versión inicial de la arquitectura da lugar a una base sólida, escalable y multiplataforma de la que irá evolucionando en futuras versiones a partir de la infraestructura tecnológica empresarial.

Link del video grupal:

https://drive.google.com/file/d/1Yzkld3M9PBMLDh9iN60PnWhdQPSzVp1O/view?usp=sharing

Link Repositorio Github:

https://github.com/cayetano-c/chat-empresarial

Referencias

Chui, M., Manyika, J., Bughin, J., Dobbs, R., Roxburgh, C., Sarrazin, H., Sands, G., & Westergren, M. (2012, 1 de julio). The social economy: Unlocking value and productivity through social technologies. McKinsey Global Institute. https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/the-social-economy

Guthkelch, A. (2025, 18 de febrero). Third of workers using risky messaging apps are a danger to employers. The HR Director. https://thehrdirector.com/business-news/employee-engagement/third-workers-using-risky-messaging-apps-danger-employers